CS310 Project Specification: GiTrip ~ *Git* for Collaborative Trip Planning ~

Tatsunori Ono

October 2025

1 Problem Statement

Trip planning is time-consuming because we must consider many factors, such as travel time, the mode of transportation, the opening and closing times of each stop, and other logistical details. The process becomes even more complicated when it comes to a group trip, as it involves messaging, sharing documents and map screenshots. When itineraries change, participants make parallel edits. It is highly likely to lose track of who changed what, when, and why, and it is difficult to "try ideas" because edits can overwrite each other. Existing tools either optimise a single itinerary (e.g., *Google Maps*) or support generic collaboration (e.g., *Notion* and *Google Docs*), but they often treat change management as an afterthought. This leads to duplicated plans, time clashes, missed opening hours, and a lack of backup options.

GiTrip addresses this problem by applying *Git*[1]-style version control to trip planning. Users initialise a *trip repository*, branch to explore alternative trip plans, and merge them with structured conflict resolution. A built-in automatic scheduler reorders stops to reduce travel time and enforces constraints such as daily active hours, per-stop stay duration, opening-hours windows, and user preferences (compact vs. sparse, morning/midday/night focus, transport mode, and so on). *GiTrip* fully visualises trip plans through two approaches. One is via an interactive timeline, which users can adjust times by dragging handles, and it automatically calculates the travel time between successive stops to keep the schedule optimised. Second is via graphs (typically a DAG — Directed Acyclic Graph) on a per-day map.

As a specific use case, a group of friends can initialise a shared repository for their London trip. Then, they can branch it so that each person can edit the trip plan as they like (e.g., museum-route and food-route). Finally, they can merge the trip plans into one by resolving conflicts, such as overlapping time, to finalise their London trip. Alternatively, users can rollback to any previous version if plans change. They can also fork and customise public trip repositories created by others.

This project is suitably challenging for a 3rd-year Computer Science project since it requires integration of Version Control System (VCS) semantics on a constraint-based scheduling with external routing services, and visualising it through an interactive UI of timeline & map editing.

2 Objectives

The aim of *GiTrip* is to turn trip planning into a clear, flexible, and easy process. The following are MoSCoW (must-have, should-have, could-have, and will-not-have right now as there is not enough time) prioritisation of each feature to accomplish the objective efficiently:

Must-haves (M)

M1 Trip repositories with history: Record of commits, branches, and all snapshot history (in JSON files). Visualisation of the trip repository history via a workflow diagram (a graph representing branches and merging). This user-friendly visualisation allows all users to intuitively use the application even if they are not familiar with *Git*.

M2 Optimised automatic scheduler: User input places, stay durations, strict-ordering of places, desired windows, daily active hours, break minutes, target day range, and user preferences. Then, output a non-overlapping plan across one or more days, considering all constraints.

M3 Routing data integration: Data for travel time will be retrieved using *Openrouteservice* and *Google Directions*, then fed into the automatic scheduler.

M4 Interactive visualisation: Trip timeline (with dotted segments to show travel gaps) and DAG map (with ordered markers with numbers). Edits commit back to the repository.

M5 Merge and conflicts: Merge operation for plan JSON (stop/field granularity) with a conflict resolver screen.

Should-haves (S)

S1 User input validation: Opening-hours validation with automatic nudge to nearest feasible slot.

S2 User preferences setting: Compact vs. sparse scheduling; morning/midday/night focus; recompute travel by mode (driving/walking/cycling/transit).

S3 Branch productivity: Create branch from Web UI, compare branches, and fork plan from a public repository.

Could-haves (C)

C1 Command-line interface (CLI) along with the Web UI: *Git*-style operations can be performed on the Web UI through clear navigation of buttons. Additionally, the `gitrip` terminal commands, such as `gitrip init/add/commit/push/branch/merge` on *GiTrip* CLI, provide the same operations as the Web UI.

C2 Public trip repository: Public repository gallery with attribution and one-click fork.

C3 Real-time tracking: Live GPS guidance during travel to detect running late, suggest reflow, and show original vs. updated times.

C4 Busy-time consideration: Take into account the crowd level in the automatic scheduler to avoid peak times.

C5 Progressive Web App (PWA): A Service Worker caches the app's core files so users can view trips offline. When offline, the app allows read-only access to the last-opened repository. Changes made offline are queued and automatically synchronised when the connection returns. Users can install the app like a native application.

Will-not-haves (W)

These are out of scope for the Minimum Viable Product (MVP) but are acknowledged and designed for future improvements.

W1 **Real-time collaborative editing:** While multiple users can work on branches and merge later, simultaneous live editing with operational transforms is out of scope.

W2 **Booking/reservation integration:** *GiTrip* plans trips but does not interface with hotel, restaurant, or ticket booking systems.

W3 **Budget tracking and expense splitting:** Financial features (cost estimates, shared expenses, currency conversion) are deferred to specialised tools for now.

W4 **Social features (chat, reviews, ratings):** Communication happens outside the app. *GiTrip* focuses on version-controlled planning, not social networking.

W5 **Mobile native apps (iOS/Android):** The PWA approach provides cross-platform access. Native Swift/Kotlin apps require resources beyond the current scope.

W6 **AI-powered itinerary generation from natural language:** For example, "plan a romantic weekend in Paris". The auto-scheduler requires structured input, and LLM integration is future work.

W7 **Learning & Smart Assistance:** While this project prioritises deterministic scheduling, there is a potential for learning user preferences (e.g., morning person, tolerance for long transfers) to bias the auto-scheduler or recommend places/routes from past trips.

2.1 Success Criteria

1. Create two alternative plans, merge, and resolve at least one timing conflict in < 5 minutes on average by 80% of 10–30 test participants.
2. Automatic scheduler produces a non-overlapping sequence of activities for ≥ 6 stops across 1–2 days with non-zero, realistic travel gaps.
3. No critical issues with accessibility and follows *Web Content Accessibility Guidelines (WCAG) 2.2*[2].

3 Technical and Concept

3.1 System Overview

- **Backend (Node.js & Express):** The server handles trip data, version control logic, and API requests. Each commit stores a snapshot with *plan.json* describing the itinerary. Using JSON for plans makes merging straightforward and avoids complex database migrations[3].
- **Frontend (EJS templates):** Server-rendered HTML pages provide the web interface. The timeline is drawn as an SVG graphic with drag handles for adjusting times. Maps use *Leaflet* (an open-source mapping library) to show numbered location pins connected by routes line, forming a DAG. When conflicts occur, users see side-by-side comparisons and choose which version to keep for each field.
- **Database (SQLite):** A lightweight file-based database stores the core data of trip repositories, branches, commits (snapshots of changes), and snapshots (the actual trip data at each commit).
- **Web Server (Nginx):** Handles incoming HTTP requests, serves static files, and forwards dynamic requests to the Node.js backend.

3.2 Place Search and Opening-hours

Place search uses *OpenStreetMap* and *Nominatim* by a server-side proxy that sets a compliant user-agent, handles requests, and returns coordinates with `opening_hours` if available. Opening-hours strings are parsed and validated. When the requested stay does not fit the daily window, the scheduler nudges to the next feasible interval or rolls forward to the next day.

3.3 Routing

If coordinates are available, *GiTrip* calls:

- ***Openrouteservice* (ORS)** for matrix endpoint[4] or per-leg (one segment of travel between two consecutive stops) routing in driving, walking, and cycling modes.
- ***Google Directions*** for transit (sequential legs).

When APIs are unavailable, haversine distance estimates minutes per leg with mode-specific min/km constants. All calls occur server-side and keys are never exposed to the browser.

3.4 Optimisation strategy and limits

Ordering uses nearest-neighbour (NN) over a duration matrix when available from ORS; otherwise haversine distance. This heuristic is $O(n^2)$ and performs well for $n \leq 50$ stops[5]. Exact TSP (Travelling Salesman Problem)[6] or meta-heuristics[7] are intentionally avoided to fit module scope and API quotas.

For large itineraries (more than 50 stops or spanning more than 10 days), *GiTrip* divides the trip into day-sized chunks, optimises each day independently, then calculates travel times between the last stop of each day and the first stop of the next.

When users manually adjust stop times by the timeline editor, those stops are locked to their chosen times. Subsequent auto-rescheduling preserves locked times and only adjusts neighbouring stops to maintain feasibility (e.g., ensuring no overlaps and respecting travel durations).

3.5 Scheduling Model

Given ordered stops $S_1, \ldots, S_n$, per-leg travel T_i (minutes from S_{i-1} to S_i), stay durations D_i, active window $[A, B]$, break b, and focus offset f:

$$a_1 = A + f, \quad d_1 = a_1 + D_1, \quad a_i = d_{i-1} + b + T_i, \quad d_i = a_i + D_i \ (i > 1).$$

If (a_i, d_i) violates desired window or opening-hours, the scheduler shifts to the nearest valid interval inside $[A, B]$; otherwise, it spills to the next day's $A + f$[8]. The user preference, compact vs. sparse, controls additional free time after each stop. The output is a list of days with ordered stops and their arrive/depart times.

3.6 Versioning and Merge

Each commit saves who made the change, a description message, links to parent commits, and a complete snapshot of the trip at that point in time. Trip plans (`plan.json`) are merged by matching stops using their unique IDs. If two people changed different aspects of the same stop (e.g., one changed arrival time, another changed duration), *GiTrip* shows both versions and asks the user to choose. The system then creates a new snapshot combining the chosen changes.

3.7 Conflict Resolution

When merging two versions of a trip, *GiTrip* compares them against their common starting point (the last version both people had before making changes). The system handles several types of conflicts:

- **Field conflicts:** When both users edit the same field, *GiTrip* shows both versions with a preview timeline for field-by-field resolution.
- **Timing conflicts:** After merging, *GiTrip* flags timing issues (overlapping stops, insufficient travel time) and offers automatic rescheduling or manual adjustment.
- **Multi-way merges:** Combined by merging pairs sequentially, with each step tracked.
- **Simultaneous edits:** If two people push changes simultaneously, the second person receives a notification and must walk through an interactive merge process before their changes can be saved.

If the merged trip is logically impossible (e.g., overlapping times, impossible travel), *GiTrip* allows the commit but marks it with an "infeasible" warning. Again, the next screen offers to automatically fix the schedule or let you adjust it manually.

3.8 Collaboration

Each repository has an owner and collaborators with roles: viewer (read), contributor (commit/push/branch), maintainer (merge/protect branches). Protected branches (e.g., `main`) require maintainer role to merge.

4 Methodology

A waterfall approach with weekly milestones and demos to the supervisor will be used. Biweekly (once in 2 weeks) in-person supervision meetings with Professor Mike Joy are scheduled throughout the project duration (Week 2, Week 4, . . . , Week 26 - excluding vacations). When needed, there will be extra meetings. For example, this time we had one extra meeting in Week 1 for checking the first draft of this project specification.

4.1 Testing

We will use the following testing for specific parts of the project:

- **Unit:** Testing for opening-hours parser, time arithmetic, merge functions, and CLI command parsing.
- **Integration:** Testing for end-to-end auto-plan with mocked routing, and push/merge conflict workflows.
- **User Acceptance Testing (UAT):** Testing for User Experience (UX) on each feature by referring to Nielsen's usability heuristics[9] as a criterion. This will be conducted through a survey with participants on Evaluation Day, measuring task completion time, error rates, and user satisfaction scores.

5 Timetable

Figure 1 shows the Gantt chart of this project, and *Figure 2* denotes the colour-coded keys for it. Some tasks are scheduled to run in parallel based on their complexity. The Christmas vacation serves as a buffer period, allowing time for coursework and personal rest. The Easter vacation falls close to the final report deadline, hence writing up and rehearsal are scheduled during this period to maintain progress.

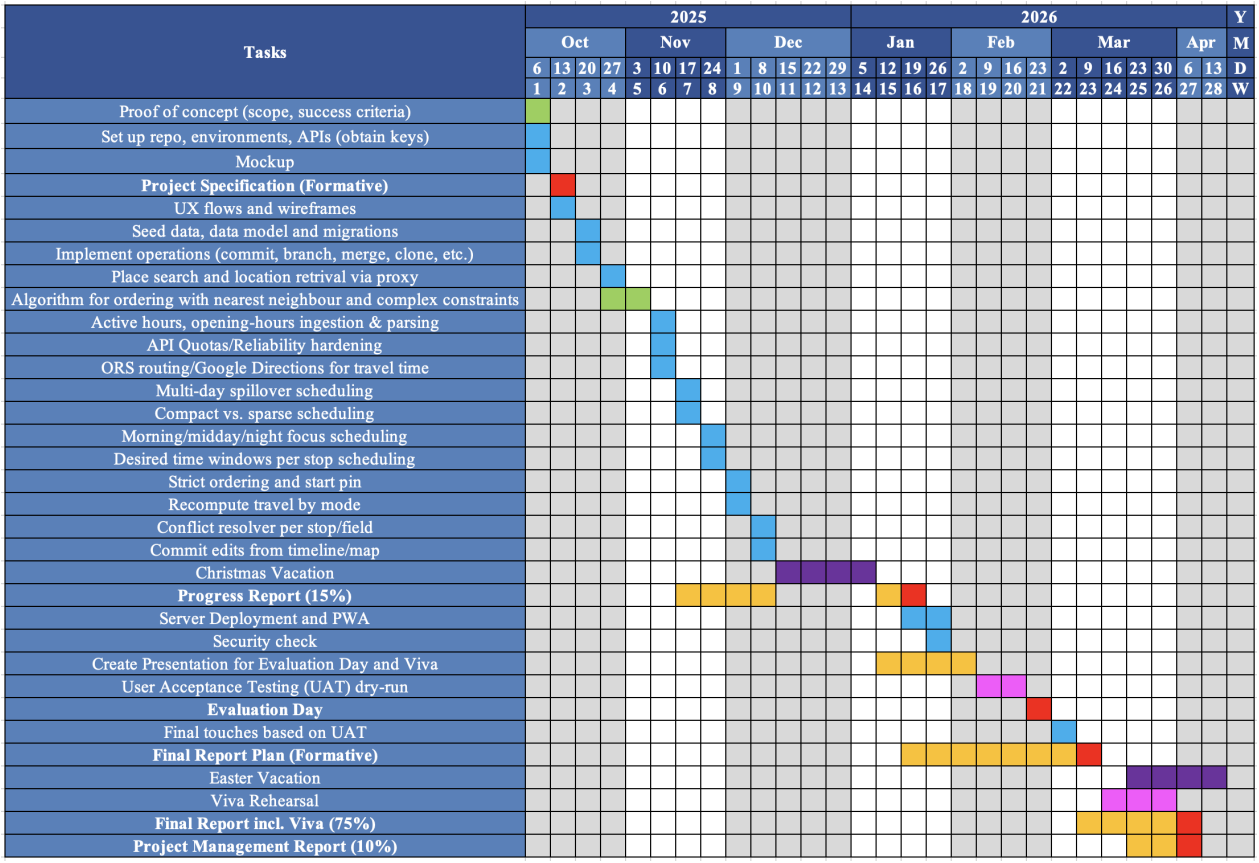

Figure 1: Project Gantt Chart

Key	
	CS310 Key Deadlines
	Planning
	Software Development
	Writing Up
	Rehearsal
	Break

Figure 2: Gantt Chart Key

6 Resources

- *OpenStreetMap/Nominatim* data for place search
- *Openrouteservice* (free) and *Google Directions* for routing
- *GitHub* for version control
- *Overleaf* for documentation
- *Vercel* (free) for deployment

6.1 Risks and Mitigations

- **API quotas and outages:** Server proxy with throttle and caching; haversine distance calculation as a fallback ensures the continuity of scheduler.
- **API key leakage:** Store keys in server env vars only; never ship to client. Rotate keys on suspected exposure; restrict by referrer/IP where supported.
- **Supervisor illness/extended absence or change of supervisor:** Maintain concise weekly status notes and a living roadmap so handover is trivial. Switch to remote supervision if needed. If unavailable for >2 weeks, ask the module organiser to assign an interim supervisor; continue against approved milestones.
- **Student illness:** Build buffers into schedule and adjust MoSCoW prioritises. If it is a serious health issue, file mitigating circumstances and agree a recovery plan at next check-in.
- **Hardware for local development lost/theft:** Manage project repository privately on *GitHub* with 2FA; push at least daily.
- **Data loss or corruption:** Commit frequently and keep a protected `main` branch.

7 Legal, Social, Ethical and Professional Issues & Considerations

GiTrip minimises personal data collection and mainly stores trip content, such as places and dates/times. Users can export or delete their data upon request, following GDPR principles. When using external services, the application follows the provider's terms and attribution requirements for *OpenStreetMap*, *Openrouteservice*, and *Google Directions*. Publicly shared plans must adhere to a brief Terms of Service, which includes rules against abusive content or unlawful use, along with a simple reporting system for inappropriate names or descriptions. For UAT, people at Evaluation Day (students and staff from the University of Warwick Department of Computer Science), my friends, and my family will be participating. Therefore, ethical consent is not required. Accessibility meets *WCAG 2.2* guidance, including keyboard operation, screen-reader labels, and adequate colour contrast.

References

[1] L. Torvalds and J. C. Hamano, "Git: distributed version control system," 2005–2025. [Online]. Available: `https://git-scm.com/`. [Accessed: 1 Jun. 2025].

[2] W3C, *Web Content Accessibility Guidelines (WCAG) 2.2*, Oct. 2023. [Online]. Available: `https://www.w3.org/TR/WCAG22/`. [Accessed: 7 Oct. 2025].

[3] J. K. Nottingham and P. Bryan, "RFC 7396: JSON Merge Patch," IETF, 2014. doi: 10.17487/RFC7396.

[4] Openrouteservice, "Matrix Endpoint — API Reference," Sept. 2025. [Online]. Available: `https://giscience.github.io/openrouteservice/api-reference/endpoints/matrix/`. [Accessed: 8 Oct. 2025].

[5] S. Hougardy, "On the nearest neighbor rule for the metric traveling salesman problem," *Discrete Applied Mathematics*, vol. 187, pp. 62–72, 2015. doi: 10.1016/j.dam.2015.03.002. [Online]. Available: `https://www.sciencedirect.com/science/article/pii/S0166218X14001486`. [Accessed: 9 Oct. 2025].

[6] D. L. Applegate, R. E. Bixby, V. Chvátal, and W. J. Cook, *The Traveling Salesman Problem: A Computational Study*. Princeton, NJ, USA: Princeton University Press, 2007. ISBN: 978-0-691-12993-8.

[7] D. S. Johnson and L. A. McGeoch, "The Traveling Salesman Problem: A Case Study in Local Optimization," in *Local Search in Combinatorial Optimization*. Chichester, UK: Wiley, 1997. [Online]. Available: `https://www.cs.ubc.ca/~hutter/previous-earg/EmpAlgReadingGroup/TSP-JohMcg97.pdf`. [Accessed: 9 Oct. 2025].

[8] E. K. Burke and H. Rudová, Eds., *Practice and Theory of Automated Timetabling VI (PATAT 2006)*. Berlin, Germany: Springer, 2007. ISBN: 978-3-540-77345-0.

[9] J. Nielsen, *Usability Engineering*. San Francisco, CA, USA: Morgan Kaufmann, 1994. ISBN: 978-0-12-518406-9.